Sesión 9

Aprendizaje No Supervisado: Clustering y Análisis de Componentes Principales

Carlos César Sánchez Coronel

MACHINE LEARNING

Contenido

1. Introducción al Aprendizaje No Supervisado
2. Análisis de Componentes Principales (PCA)
3. K-Means Clustering
4. DBSCAN (Density-Based Clustering)
5. Clustering Jerárquico
6. Métricas de Evaluación para Clustering
7. Caso Integrado: Segmentación de Clientes
8. Implementación en Python
9. Conexión y Resumen

Contenido

1. Introducción al Aprendizaje No Supervisado
2. Análisis de Componentes Principales (PCA)
3. K-Means Clustering
4. DBSCAN (Density-Based Clustering)
5. Clustering Jerárquico
6. Métricas de Evaluación para Clustering
7. Caso Integrado: Segmentación de Clientes
8. Implementación en Python
9. Conexión y Resumen

¿Qué es el aprendizaje no supervisado?

- Trabajamos con datos **sin etiquetas**.
- Objetivos principales:
  - Descubrir grupos o segmentos naturales (**clustering**).
  - Reducir la dimensionalidad para visualización, compresión o preprocesamiento (**PCA**).
  - Detectar anomalías o valores atípicos.
- No hay una métrica de rendimiento única; la evaluación depende del objetivo y del conocimiento del dominio.

Contenido

1. Introducción al Aprendizaje No Supervisado
2. Análisis de Componentes Principales (PCA)
3. K-Means Clustering
4. DBSCAN (Density-Based Clustering)
5. Clustering Jerárquico
6. Métricas de Evaluación para Clustering
7. Caso Integrado: Segmentación de Clientes
8. Implementación en Python
9. Conexión y Resumen

PCA: fundamento matemático

Objetivo

Proyectar los datos en un espacio de menor dimensión maximizando la varianza retenida.

1. Centrar la matriz de datos $\mathbf{X}$ (restar media por columna).
2. Calcular matriz de covarianza: $\mathbf{\Sigma} = \dfrac{1}{n-1}\mathbf{X}^T\mathbf{X}$.
3. Obtener autovalores $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_p$ y autovectores asociados $\mathbf{v}_1, \ldots, \mathbf{v}_p$.
4. El k-ésimo componente principal es $\mathbf{X}\mathbf{v}_k$.

Interpretación de componentes

- **Autovalores**: representan la varianza explicada por cada componente.
- **Proporción de varianza explicada**: $\frac{\lambda_j}{\sum_{i=1}^{p} \lambda_i}$.
- **Loadings** (autovectores): indican la contribución de cada variable original al componente.
- Un valor alto en magnitud señala variables importantes para ese componente.

Scree plot y elección del número de componentes

Scree plot: gráfico de autovalores en orden descendente.

- Se busca un "codo" donde la caída se vuelve marginal.
- Se retienen los componentes hasta el codo.

Ejemplo numérico

Autovalores: [4.5, 2.1, 1.3, 0.8, 0.6, 0.4, 0.3, 0.2, 0.1, 0.05]
Varianza total = 10.35.
Primer componente: 43.5 %, dos primeros: 63.8 %. El codo sugiere k=3.

Aplicaciones de PCA

- **Visualización**: proyectar datos a 2D o 3D para inspección.
- **Reducción de ruido**: descartar componentes de baja varianza (ruido).
- **Preprocesamiento**: reducir dimensionalidad antes de modelos sensibles (KNN, clustering).
- **Compresión**: almacenar solo las primeras componentes.

Contenido

1. Introducción al Aprendizaje No Supervisado
2. Análisis de Componentes Principales (PCA)
3. K-Means Clustering
4. DBSCAN (Density-Based Clustering)
5. Clustering Jerárquico
6. Métricas de Evaluación para Clustering
7. Caso Integrado: Segmentación de Clientes
8. Implementación en Python
9. Conexión y Resumen

Algoritmo de K-Means

1. Inicializar k centroides (aleatorio o K-Means++).
2. Asignar cada punto al centroide más cercano (distancia euclidiana).
3. Recalcular cada centroide como la media de los puntos asignados.
4. Repetir 2 y 3 hasta convergencia (centroides no cambian o máximo de iteraciones).

Función objetivo (inercia)

$$J = \sum_{j=1}^{k} \sum_{x \in C_j} \|x - \mu_j\|^2$$

Minimiza la suma de distancias cuadráticas intra-cluster.

Elección del número de clusters k

Método del codo

- Graficar inercia vs k.
- Elegir k donde la mejora se vuelve marginal.

Coeficiente de silueta

- Mide cohesión y separación.
- Promedio para cada k; se elige el k con mayor valor.

Ejemplo (Pregunta 5)

Inercia: [1000,600,450,400,380,370,365,360,358]
Silueta: [0.35,0.48,0.52,0.50,0.47,0.44,0.41,0.38,0.35]
Codo sugiere $k = 3$ o 4; silueta máxima en $k = 3$. Se elige $k = 3$.

Inicialización: K-Means++

- K-Means es sensible a la inicialización (óptimos locales).
- **K-Means++** elige centroides dispersos:
  1. Primer centroide aleatorio.
  2. Para cada punto, calcular distancia al centroide más cercano ya elegido.
  3. Elegir siguiente centroide con probabilidad proporcional al cuadrado de esa distancia.
  4. Repetir hasta tener k centroides.
- Es la implementación por defecto en scikit-learn.

Limitaciones de K-Means

- Asume clusters **esféricos** y de tamaño similar.
- Sensible a **outliers**.
- Requiere especificar k.
- Puede converger a **óptimos locales**.

Aplicaciones típicas

Segmentación de clientes, compresión de imágenes, agrupamiento de documentos.

Contenido

1. Introducción al Aprendizaje No Supervisado
2. Análisis de Componentes Principales (PCA)
3. K-Means Clustering
4. DBSCAN (Density-Based Clustering)
5. Clustering Jerárquico
6. Métricas de Evaluación para Clustering
7. Caso Integrado: Segmentación de Clientes
8. Implementación en Python
9. Conexión y Resumen

DBSCAN: conceptos básicos

Parámetros

- ε (eps): radio de vecindad.
- `minPts`: número mínimo de puntos para formar una región densa.

Clasificación de puntos

- **Punto núcleo**: tiene al menos `minPts` vecinos dentro de ε.
- **Punto borde**: no es núcleo pero es vecino de un núcleo.
- **Ruido**: no es núcleo ni borde.

Ventajas y desventajas de DBSCAN

Ventajas

- No requiere k previo.
- Detecta clusters de forma arbitraria.
- Identifica outliers como ruido.

Desventajas

- Sensible a ε y `minPts`.
- Difícil de ajustar si las densidades varían.

Aplicaciones

Detección de anomalías, datos geoespaciales, formas no convexas.

Contenido

1. Introducción al Aprendizaje No Supervisado
2. Análisis de Componentes Principales (PCA)
3. K-Means Clustering
4. DBSCAN (Density-Based Clustering)
5. Clustering Jerárquico
6. Métricas de Evaluación para Clustering
7. Caso Integrado: Segmentación de Clientes
8. Implementación en Python
9. Conexión y Resumen

Clustering jerárquico aglomerativo

1. Comenzar con cada punto como un cluster individual.
2. En cada paso, fusionar los dos clusters más cercanos según una medida de **enlace**.
3. Repetir hasta un solo cluster.

Tipos de enlace

- **Single linkage**: distancia mínima entre puntos $\rightarrow$ forma cadenas.
- **Complete linkage**: distancia máxima $\rightarrow$ clusters compactos.
- **Average linkage**: distancia promedio.
- **Ward**: minimiza el incremento de varianza intra-cluster.

Dendrograma y elección de clusters

Dendrograma

- Representa la jerarquía de fusiones.
- Cortando a cierta altura se obtienen clusters.
- La altura del corte se elige según la distancia de fusión (buscar un salto grande).

- Ventaja: no requiere k previo, jerarquía interpretable.
- Desventaja: costoso computacionalmente ($O(n^3)$ o $O(n^2)$).

Contenido

1. Introducción al Aprendizaje No Supervisado
2. Análisis de Componentes Principales (PCA)
3. K-Means Clustering
4. DBSCAN (Density-Based Clustering)
5. Clustering Jerárquico
6. Métricas de Evaluación para Clustering
7. Caso Integrado: Segmentación de Clientes
8. Implementación en Python
9. Conexión y Resumen

Coeficiente de silueta

Para cada punto i:

- $a(i)$: distancia media a los puntos de su mismo cluster (cohesión).
- $b(i)$: distancia media al cluster vecino más cercano (separación).

$$s(i) = \frac{b(i) - a(i)}{\text{máx}\{a(i), b(i)\}}$$

- $s(i) \approx 1$: cluster compacto y bien separado.
- $s(i) \approx 0$: solapamiento.
- $s(i) < 0$: asignación incorrecta.

Se promedia sobre todos los puntos.

Índice de Davies-Bouldin

- Mide la similitud promedio entre cada cluster y su más similar.
- Para cada cluster i, se define $R_i = \text{máx}_{j \neq i} \frac{s_i + s_j}{d_{ij}}$, donde s_i es la dispersión intra-cluster (distancia media al centroide) y d_{ij} es la distancia entre centroides.
- Índice final $= \frac{1}{k} \sum_{i=1}^{k} R_i$.
- **Valores más bajos indican mejor separación**.

Comparación de algoritmos de clustering

Algoritmo	**Forma clusters**	**Número de clusters**	**Ruido**
K-Means	Esféricos	Fijo k	No
DBSCAN	Arbitraria	Automático	Sí
Jerárquico	Cualquier	Corte en dendrograma	No

Contenido

1. Introducción al Aprendizaje No Supervisado
2. Análisis de Componentes Principales (PCA)
3. K-Means Clustering
4. DBSCAN (Density-Based Clustering)
5. Clustering Jerárquico
6. Métricas de Evaluación para Clustering
7. Caso Integrado: Segmentación de Clientes
8. Implementación en Python
9. Conexión y Resumen

Problema y datos

- Empresa de retail con 5000 clientes.
- Variables: monto total gastado, frecuencia de compra, antigüedad, edad.
- Objetivo: segmentar para personalizar campañas de marketing.

Pipeline de segmentación

1. **Preprocesamiento**: estandarización (Z-score) de variables numéricas.
2. **PCA**: reducir dimensionalidad para visualización (elegir componentes que expliquen al menos 75 % de varianza).
3. **K-Means**: probar k de 2 a 10, usar método del codo y silueta.
4. **Seleccionar k óptimo** (ej. $k = 4$).
5. **Caracterizar clusters** mediante perfiles medios en variables originales.

Resultados y acciones de marketing

Cluster	Edad	Ingresos	Frecuencia
0	Baja	Altos	Alta
1	Media-alta	Bajos	Baja
2	Alta	Medios	Baja
3	Baja	Bajos	Media

- Cluster 0: ofertas exclusivas, productos premium.
- Cluster 1: promociones para incentivar compras.
- Cluster 2: productos adaptados a mayores.
- Cluster 3: descuentos para jóvenes.

Contenido

1. Introducción al Aprendizaje No Supervisado
2. Análisis de Componentes Principales (PCA)
3. K-Means Clustering
4. DBSCAN (Density-Based Clustering)
5. Clustering Jerárquico
6. Métricas de Evaluación para Clustering
7. Caso Integrado: Segmentación de Clientes
8. Implementación en Python
9. Conexión y Resumen

PCA con scikit-learn

```python
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

print("Varianza explicada:", pca.explained_variance_ratio_)
```

K-Means y selección de k

```python
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

inertias, silhouettes = [], []
for k in range(2, 11):
    kmeans = KMeans(n_clusters=k, random_state=42)
    labels = kmeans.fit_predict(X_scaled)
    inertias.append(kmeans.inertia_)
    silhouettes.append(silhouette_score(X_scaled, labels))

# Elegir k con mayor silueta
best_k = np.argmax(silhouettes) + 2
kmeans = KMeans(n_clusters=best_k, random_state=42)
labels = kmeans.fit_predict(X_scaled)
```

DBSCAN y clustering jerárquico

```
from sklearn.cluster import DBSCAN

dbscan = DBSCAN(eps=0.5, min_samples=5)
labels_db = dbscan.fit_predict(X_scaled)  # -1 indica ruido

# Clustering jer rquico y dendrograma
from scipy.cluster.hierarchy import dendrogram, linkage
import matplotlib.pyplot as plt

Z = linkage(X_scaled, method='ward')
plt.figure(figsize=(10,5))
dendrogram(Z)
plt.show()
```

Contenido

1. Introducción al Aprendizaje No Supervisado
2. Análisis de Componentes Principales (PCA)
3. K-Means Clustering
4. DBSCAN (Density-Based Clustering)
5. Clustering Jerárquico
6. Métricas de Evaluación para Clustering
7. Caso Integrado: Segmentación de Clientes
8. Implementación en Python
9. Conexión y Resumen

Conexión con el resto del curso

- PCA se usa como preprocesamiento en modelos supervisados para reducir dimensionalidad y ruido.
- Los clusters pueden servir como nuevas características en modelos supervisados.
- DBSCAN es base para detectores de anomalías.
- La interpretación de clusters complementa el análisis exploratorio.

Lo que hemos aprendido

Conceptos clave

- **PCA**: reducción de dimensionalidad, autovalores, scree plot.
- **K-Means**: algoritmo, inercia, método del codo, silueta, K-Means++.
- **DBSCAN**: densidad, puntos núcleo/borde/ruido, parámetros.
- **Clustering jerárquico**: aglomerativo, enlaces, dendrograma.
- **Métricas**: silueta, Davies-Bouldin.
- **Caso práctico**: segmentación de clientes.

¡Gracias!